Who should the bank call?

Dataset: **Bank Marketing** (UCI, OpenML id 1461)

Khaled Elnagar Alzira Ferreira

MSc, *Supervised Learning & Model Engineering*, Albert School, October 2026

Code: `https://github.com/khaledngr/bank-marketing-project`

Abstract

A bank phones clients to sell a term deposit; 11.7% of 45,211 clients subscribe. A call costs EUR 5, a subscription earns EUR 50. We built a leakage-free pipeline from raw data to a served prediction endpoint. A tuned LightGBM reaches an average precision (AP) of 0.476 on an untouched test set (random: 0.117). Calling the 24% of clients above a profit-optimal threshold reaches 67% of subscribers and earns EUR 2,711 per 1,000 clients, 3.2× more than calling everyone. Probabilities are well calibrated (ECE 0.010), but a time-ordered check shows they go stale when the base rate shifts. Age-group thresholds shrink the measured equal-opportunity gap from 35 to 5 points at a cost of 6% of profit.

1 Problem framing and data

- **Task:** before calling, predict $p = P(\text{subscribe})$ from what the bank already knows (profile, balance, loans, contact history).
- **Decision rule:** call if $p \times 50 > 5$, i.e. $p > 0.10$. Costs are our assumptions (`config.yaml`).
- **Data:** 45,211 clients, 16 features; target `Class`= 2 mapped to $y = 1$ (checked against the documented 11.7%). No missing cells, no duplicates; missing information hides in `unknown` (`contact`: 29%) and `pdays`= −1 ("never contacted", 82%).
- **Split:** stratified 80/20 with a fixed seed. Test set (9,043) scored once at the end; validation = stratified 5-fold CV on the 80%, same folds for every model. EDA on training data only.

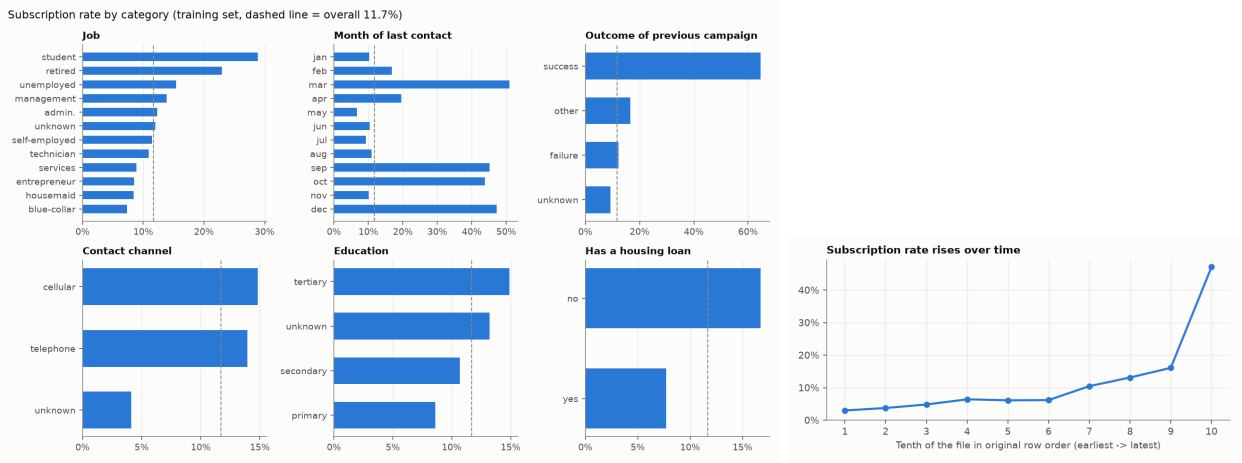


Figure 1: Left: subscription rate by category (dashed = 11.7%). Right: rate along the file, which UCI documents as ordered by date (May 2008 – Nov 2010).

Decision. Drop `duration` (call length) before any split.
Why. It is only known after the call (calls < 90 s convert at 0.7%, > 6 min at 33%): pure leakage.
Risk. Time drift: the rate rises from 3% to 47% along the file and the split is random, so the test score describes clients from the same period, not a future campaign (quantified by a time-ordered check, Section 9). No client ID: repeated clients across the split cannot be ruled out.

2 Feature engineering

Inside the Pipeline, refitted in every fold: median imputation + scaling for numbers, one-hot encoding for categories (`unknown` kept as a category: `contact`=unknown converts at 4% vs. 15% for cellular). Three rule-based features (no fitted statistics, so no leakage):

- `contacted_before`: flags `pdays`$= -1$, which is then imputed instead of read as "1 day ago".
- Logarithms of `balance` (signed), `campaign`, `previous`: long tails (balance up to EUR 102,127).
- `life_stage` (< 25 / 25–59 / ≥ 60): subscription is U-shaped in age (26%, 10%, 34%).

Features (cumulative)	Logistic regression AP	LightGBM AP
Raw columns	0.402	0.445
+ `contacted_before`	0.402	0.449
+ logarithms	0.406	0.447
+ `life_stage`	0.408	0.449

Table 1: Feature ablation, mean AP over 5 CV folds (fold std ≈ 0.01).

Decision. Keep the three features: cheap, interpretable, and they never lower AP.
Why. Linear model gains +0.006 AP (it cannot bend on its own); trees gain nothing measurable (they find thresholds and U-shapes themselves).
Risk. Gains are within fold noise. `life_stage` derives from age, the sensitive attribute (Section 8).

3 Class imbalance

Accuracy is uninformative ("never call" is 88% accurate). Metrics: **AP** (ranking of the rare class; random = 0.117), **precision** (calls that convert), **recall** (subscribers reached) and **profit**. Remedies compared on LightGBM inside the folds; class weights computed per training fold, SMOTE inside an `imblearn` Pipeline. Thresholds chosen on out-of-fold scores.

Remedy	Threshold	AP	Precision	Recall	Called	Profit / 1,000
None	0.50 (default)	0.449	0.62	0.25	5%	EUR 1,213
None	**0.11 (profit)**	**0.449**	0.32	0.65	24%	**EUR 2,638**
Class weights	0.44 (profit)	0.446	0.29	0.67	27%	EUR 2,606
SMOTE	0.16 (profit)	0.432	0.29	0.66	27%	EUR 2,512
Call everyone	–	0.117	0.12	1.00	100%	EUR 849

Table 2: Imbalance remedies (default LightGBM, 5-fold CV).



Figure 2: Left: precision–recall curves. Right: profit per 1,000 clients by threshold (dot = best; dashed = call everyone).

Decision. Cost-based threshold (0.11); no class weights, no SMOTE.
Why. Weights and SMOTE do not improve AP and both distort probabilities: mean prediction 36% with weights (ECE 0.24) and 16% with SMOTE (ECE 0.04), vs. 11.6% (ECE 0.006) plain, for 11.7% observed. SMOTE also interpolates one-hot columns (SMOTENC would not, but SMOTE already loses). The threshold doubles profit vs. 0.5 and lands next to the break-even 0.10.
Risk. Threshold chosen on the same out-of-fold scores we report (checked on test). Profit depends on the assumed costs; the profit curve is flat between 0.09 and 0.15 (Figure 2).

4 Models, Pipeline and tuning

One scikit-learn Pipeline, used for CV, tuning, explanation and serving: *features* → *ColumnTransformer* → *model*. Refitting the whole object per fold means no statistic is learned from validation clients. Tuning: Optuna (TPE, seed 42), 30 trials per model, each scored by mean AP on the same 5 folds; search spaces in Appendix A. All runs are tracked in MLflow (Section 5).

	AP		Profit / 1,000		Tuned, at profit threshold		
Model	default	tuned	default	tuned	Thr.	Prec.	Recall
Call everyone	0.117	–	EUR 849	–	–	0.12	1.00
Logistic regression (baseline)	0.408	0.409	EUR 2,347	EUR 2,366	0.13	0.30	0.61
Random forest	0.412	0.448	EUR 2,525	EUR 2,613	0.12	0.32	0.65
LightGBM	**0.449**	**0.459**	EUR 2,638	**EUR 2,676**	0.11	0.32	0.66

Table 3: Three models, same folds, same metrics (5-fold CV; AP fold std 0.010–0.017).

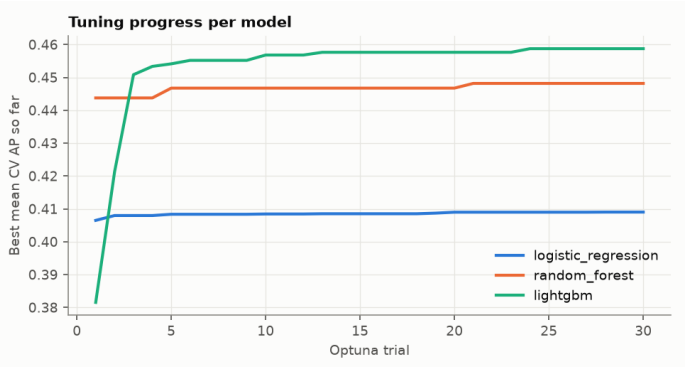


Figure 3: Optuna: best mean CV AP so far per trial; most of the gain comes in the first 10–20 trials.

What each step bought.

- No model → logistic regression: AP 0.117 → 0.408, profit ×2.8.
- → random forest: finds interactions; gains most from tuning (+0.036 AP: min leaf 4, 20% of features per split, i.e. less overfit, more diverse trees).
- → LightGBM: best AP and profit; tuned to learning rate 0.013 with 560 trees.
- Logistic regression barely tunes (+0.001): limited by its linear form; Optuna leaned to Lasso (`l1_ratio` 0.85).

Decision. Tuned LightGBM as final model.
Why. Best AP, ROC-AUC and profit; the course's working model.
Risk. Its lead over a tuned random forest (≈ 0.011 AP) is about one fold std. The same folds select and score, so we checked with nested CV (`src/nested_cv.py`: 5 outer folds, 15 Optuna trials and the threshold chosen inside each): AP 0.458 ± 0.012 and EUR 2,667 vs. 0.459 and EUR 2,676, threshold 0.10–0.11 in every fold, so the selection bias is negligible. Tree count was tuned instead of early stopping, which would need an extra split inside each fold.

5 Experiment tracking

MLflow (local SQLite store). One parent run per experiment with its tables and figures as artifacts; one child run per candidate with all parameters, metrics, per-fold AP and the git commit. Covered: feature ablation, imbalance remedies, model comparison, 90 Optuna trials (plus each study's trial table), explainability, calibration, fairness, the final model (logged as an MLflow model) and the two supplementary checks. One seed drives every split, model and sampler, so re-running the scripts should reproduce the same metrics (run IDs and timings differ). The store itself (`mlflow.db`) is not committed, so a reader rebuilds it by running the scripts (≈ 20 min); Optuna trials carry parameters and metrics, artifacts are kept per study.



Figure 4: All 90 Optuna trials, read back from the MLflow store (`python -m src.tracking`): mean CV AP per trial, bar = fold standard deviation.

6 Explainability

Tuned LightGBM fitted on 4 folds, explained on the 5th (7,234 unseen clients): same settings as the deployed model but not the deployed fit, so the test set stays untouched. Three global views: **gain** (training improvement), **permutation** (validation AP lost when shuffled; 10 repeats; `pdays`/`previous`/`poutcome` shuffled together, `contacted_before` follows as it is derived from `pdays` inside the Pipeline) and **mean |SHAP|** (log-odds contribution).

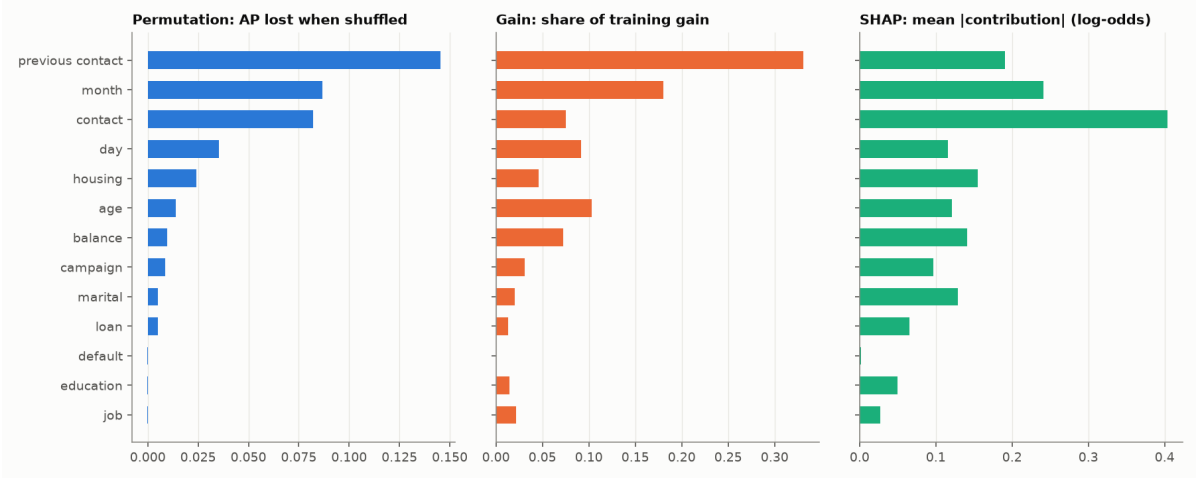


Figure 5: Global importance, aggregated to client variables.

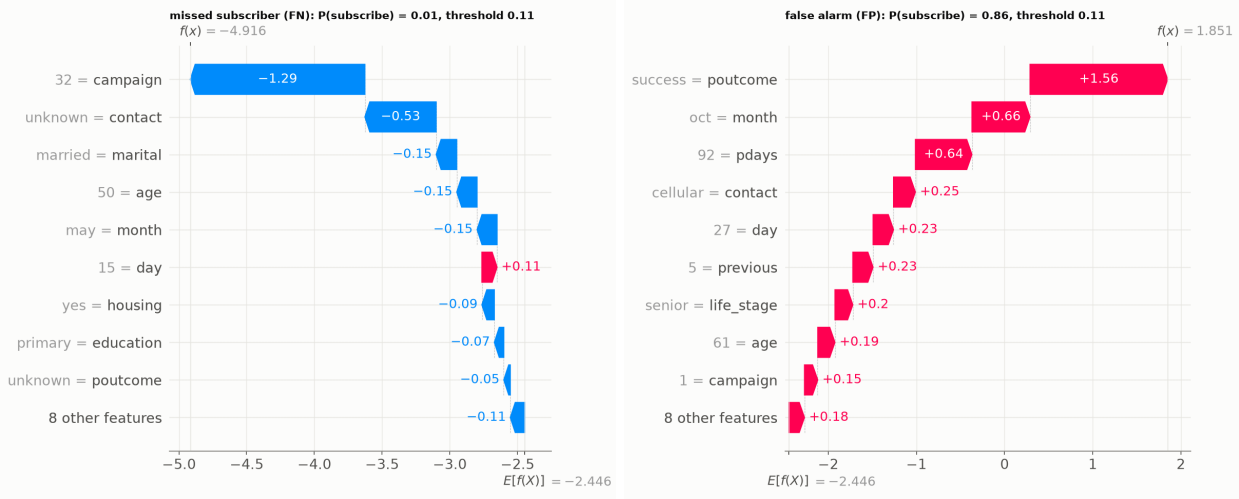


Figure 6: Local SHAP (log-odds). Left: missed subscriber, $p = 0.007$, already called 32 times. Right: false alarm, $p = 0.86$, retiree with a previous success.

Business sense.

- Previous-contact information dominates (AP −0.145 when its three raw columns are shuffled as one group, so not directly comparable with single variables): past buyers buy again. ✓
- Housing loan ↓, higher balance ↑, many calls this campaign ↓, age U-shape. ✓
- Job, education, default ≈ 0: shuffling them costs no AP once the other variables are known; we did not test whether they carry signal on their own (e.g. job = retired overlaps with age).
- `month`, `contact` and `day` partly encode *when* the call happened (drift): `contact`=unknown fills the first 28% of the file (2008) and almost nothing after, and `day` (day of month) has no client meaning. Treat these as timing, not behaviour.
- Views disagree as expected: `contact` is #1 by SHAP but #3 by permutation (`month` substitutes when it is shuffled); age is high by gain (training-based, favours continuous features) but low by permutation.

Decision. Report all three views and four local cases (two more in Appendix B).
Why. Each view answers a different question; local cases show the model's logic, e.g. "stop after 32 calls" is rational at EUR 5 per call.
Risk. SHAP explains the model, not causes. Correlated features share or swap importance.

7 Calibration

The call rule needs honest probabilities. Compared on out-of-fold scores: uncalibrated, Platt (sigmoid) and isotonic, the latter two via `CalibratedClassifierCV` on inner folds (no calibrator sees its own training labels).

Candidate	AP	Brier	ECE (10 bins)	Mean predicted	Profit / 1,000
Uncalibrated (CV)	0.459	0.0811	0.006	11.6%	EUR 2,676
Platt (CV)	0.458	0.0811	0.006	11.7%	EUR 2,667
Isotonic (CV)	0.458	0.0810	0.003	11.7%	EUR 2,651
Uncalibrated (test)	0.476	0.0798	0.010	–	EUR 2,711

Table 4: Probability quality (observed rate 11.7%).

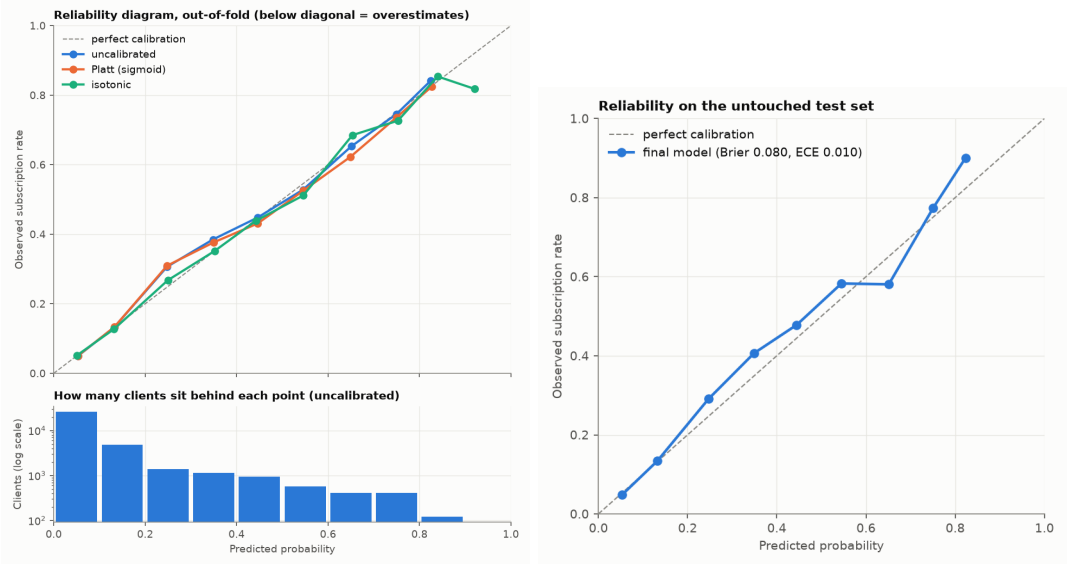


Figure 7: Reliability diagrams. Left: CV with clients per bin. Right: test set.

Decision. Keep the model uncalibrated; trust its probabilities for the call rule.
Why. Already calibrated: in the 10–20% bin, where decisions are made, predicted = observed = 13.2%. Brier 0.081 vs. 0.103 for always predicting 11.7% (skill score 0.22). Slightly under-confident at 20–40% (predicts 25%, observes 31%). Isotonic halves ECE but not Brier and lowers profit: simpler wins. Log-loss training and no reweighting (Section 3) keep it honest.
Risk. Calibration goes stale under drift; group-level calibration differs (ECE 0.010–0.049 per age group on test).

8 Fairness audit on age

- **Groups:** young (< 25), working (25–59), senior (≥ 60): the U-shape of the data, same cut-offs as `life_stage`.
- **Definition: equal opportunity** (equal recall). Harm = an interested client never offered the product. Demographic parity does not fit: interest differs by age (26% / 10% / 34%), and with different base rates the definitions cannot all hold.
- **Audit (CV):** recall 0.90 / 0.61 / 0.97, 95% Wilson intervals do not overlap. Removing age barely helps (gap 36 $\rightarrow$ 29 points; `job` = retired/student are proxies) and worsens senior calibration (ECE 0.027 $\rightarrow$ 0.056).
- **Mitigation (post-processing):** per-group thresholds set on out-of-fold scores to reach the overall recall (0.66): 0.34 / 0.09 / 0.33. Model unchanged. Frozen, then applied to the test set:

Test set	Group	Subscribers	Called	Recall [95% CI]	Unwanted calls	Precision
One threshold (0.11)	young	39	68%	0.95 [0.83, 0.99]	58%	0.36
	working	907	21%	0.62 [0.59, 0.65]	16%	0.31
	senior	112	82%	0.97 [0.92, 0.99]	75%	0.39
Age-group thresholds	young	39	30%	0.64 [0.48, 0.77]	19%	0.54
	working	907	29%	0.69 [0.66, 0.72]	24%	0.25
	senior	112	42%	0.69 [0.60, 0.77]	29%	0.54

Table 5: Fairness on the test set. Recall gap 35 → 5 points; profit EUR 2,711 → EUR 2,551 per 1,000 clients (−6%).

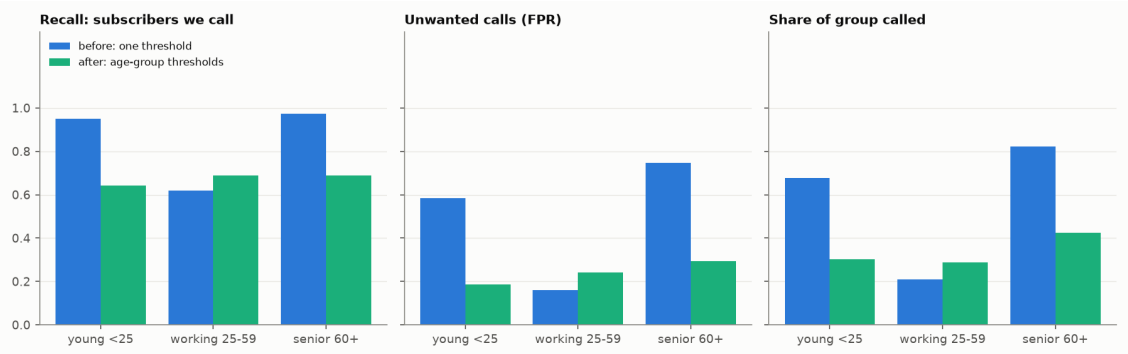


Figure 8: Test set, per age group: recall, unwanted calls and share called, before and after mitigation.

Decision. Report both policies; the API returns both decisions.
Why. Mitigation shrinks the measured recall gap from 35 to 5 points; the group intervals now overlap, which means no gap is detectable, not that the groups are proven equal. Fewer unwanted calls to seniors (75% → 29%) is a side effect, not the criterion.
Risk. −6% profit; levelling down: the gap closes mostly by calling fewer young and senior clients, which skips calls worth +EUR 11.5 at $p = 0.33$; precision gap grows (0.25 vs. 0.54), i.e. predictive parity breaks; only 39 young subscribers in test (recall CI 0.48–0.77), so the small gap is not statistically established; using age explicitly is disparate treatment, a legal choice for the bank (EU).

9 Final evaluation and deployment

All choices frozen on CV; Pipeline refitted on the 36,168 training clients; test set scored once.

	CV	**Test**	Test: call everyone
AP / ROC-AUC	0.459 / 0.802	**0.476 / 0.809**	0.117 / 0.500
Precision / recall at 0.11	0.32 / 0.66	0.33 / 0.67	0.12 / 1.00
Clients called	24%	24%	100%
Profit per 1,000 clients	EUR 2,676	**EUR 2,711**	EUR 850
Brier / ECE	0.081 / 0.006	0.080 / 0.010	–

Table 6: Final model on the untouched test set (9,043 clients).

Served endpoint. FastAPI app loading the saved Pipeline and thresholds. `POST /predict` takes one client's raw fields (validated categories, age 18–100, no `duration`) and returns p, the expected value $50p - 5$, and the business and age-fair decisions. Raw inputs go through the same fitted preprocessing as in training.

Decision. Ship the tuned LightGBM with threshold 0.11 behind the API.
Why. Test matches CV (AP 0.476 vs. 0.459, within fold noise); 3.2× the profit of calling everyone; one call in three converts vs. one in eight at random.
Risk. Same-period test set; proof of concept without authentication, batch scoring or monitoring of drift and calibration.

Supplementary time-ordered check (`src/temporal.py`; the main test is untouched). Trained on the first 80% of the file (yes 6.7%), scored on the last 20% (yes 31.6%): the model still ranks (AP 0.487 vs. 0.316 by chance, ROC-AUC 0.69), but the lift falls from 4.1× to 1.5×, probabilities lag the new base rate (mean 16% vs. 32% observed, ECE 0.16), and calling everyone would earn more (EUR 10,791 vs. EUR 7,291 per 1,000). Under drift the model must be recalibrated and its threshold re-checked each campaign. Caveat: the hyperparameters come from the random-split tuning, which saw late rows, so this check is, if anything, slightly optimistic.

10 Conclusion

A calibrated, explainable LightGBM with a cost-based threshold roughly triples the profit of an untargeted campaign; its age gap is measurable and can be reduced at a quantified cost. The time-ordered check shows the main limit: when the base rate shifts, probabilities go stale, so the model needs recalibration and monitoring per campaign before real use.

Contribution statement

How we worked. Most of the project was built in joint sessions on one laptop (Khaled's), with Claude Code as assistant; we discussed and decided each step together. Because the work was shared, we split commit authorship between us afterwards, so the Git author field records that split, not who typed each line; this is also why commit times cluster.

Who did what. We did every step together (requirements 1–10, final test, API, report); no step was done by one of us alone, and each of us can explain every part.

Use of AI assistants

We used Claude Code (Anthropic) as a coding assistant: it drafted code and text under our direction; we reviewed, ran and validated every step and made the final choices.

A Hyperparameters

Model	Search space (30 trials)	Chosen
Logistic regression	$C \in [10^{-3}, 10]$ (log), `l1_ratio` $\in [0, 1]$, saga	0.081, 0.85
Random forest	trees 100–500, depth 4–30, min leaf 1–100 (log), max features 0.1–0.8	232, 19, 4, 0.20
LightGBM	trees 100–1000, learning rate 0.01–0.2 (log), leaves 8–128 (log), min child samples 5–200 (log), subsample 0.5–1, colsample 0.4–1, L2 penalty 10^{-3}–10 (log)	560, 0.013, 37, 17, 0.87, 0.55, 1.25

B Additional figures

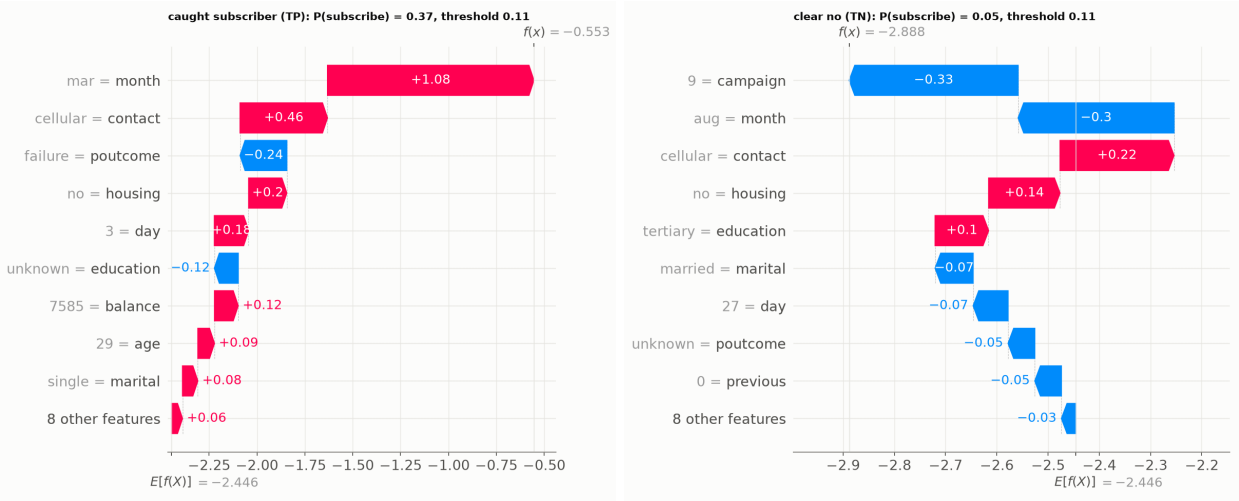


Figure 9: Local SHAP. Left: caught subscriber (March, mobile). Right: clear "no" (9 calls, August).

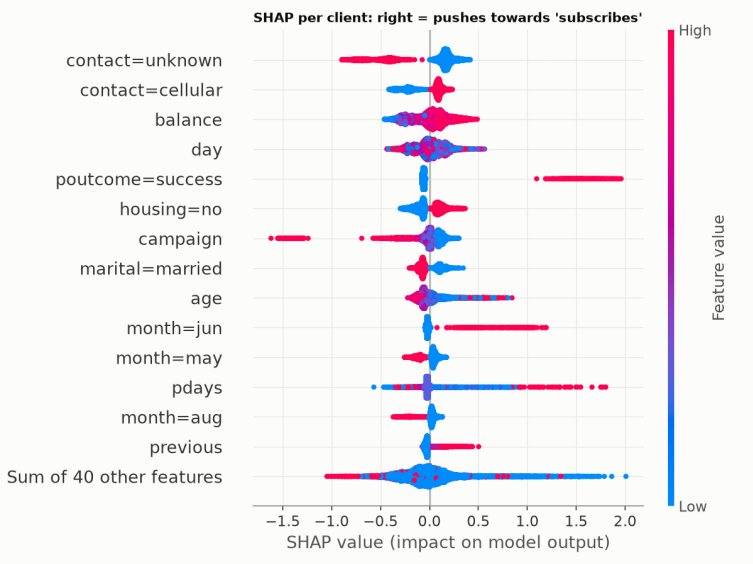


Figure 10: SHAP beeswarm: one dot per client, colour = feature value.

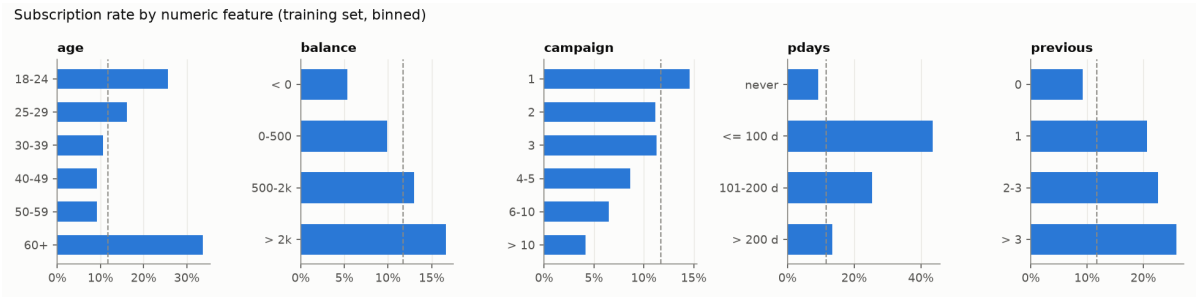


Figure 11: Subscription rate by binned numeric feature (training set).

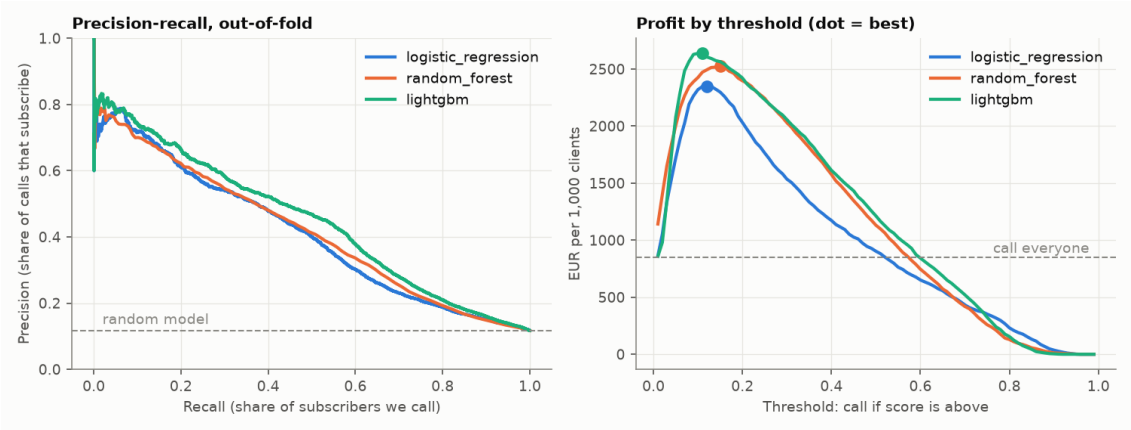


Figure 12: Precision–recall and profit curves of the three default models (CV).

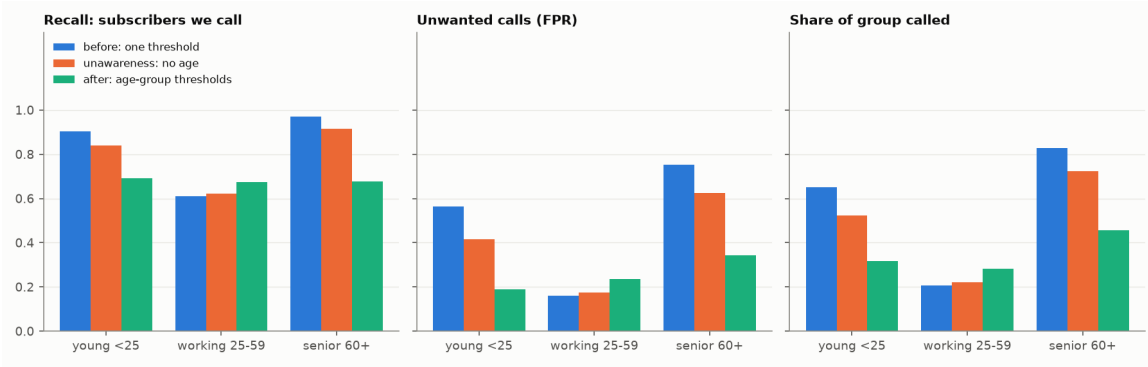


Figure 13: Fairness on CV: one threshold, unawareness (no age), age-group thresholds.